



Politecnico
di Torino

Algorithms and Programming

Iniziato	martedì, 21 dicembre 2021, 17:10
Stato	Completato
Terminato	martedì, 21 dicembre 2021, 17:11
Tempo impiegato	9 secondi
Valutazione	0,00 su un massimo di 20,00 (0%)

Domanda 1

Risposta non data

Punteggio max.:
2,00

The main program calls function f as f(3, 5). In turn function f calls function g.

```
void f (int n, int m) {  
    printf ("F");  
    if (n<=0)  
        return;  
    if (n%2==0) {  
        g(n,m-1);  
    } else {  
        f(n-1,m);  
    }  
    return;  
}  
  
void g (int n, int m) {  
    printf ("G");  
    if (m<=0)  
        return;  
    if (m%2==0) {  
        g(n,m-1);  
    } else {  
        f(n-1,m);  
    }  
    return;  
}
```

Following the recursion tree of f and g, indicate the number of times the character 'F' and the character 'G' are displayed on standard output.

Please, report your response as a pair of integer values (the first one for 'F', the second one for 'G'). No other symbols must be included in the response. For example, if the function displays "FGFGFGGG", you must reply: 3 5

Risposta:



La risposta corretta è : 4 2

Domanda 2

Risposta non data

Punteggio max.:

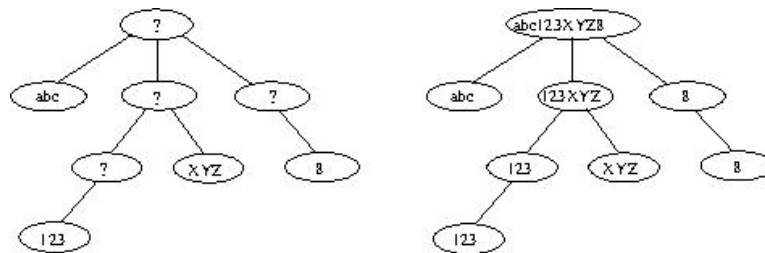
4,00

A n-ary tree is organized adopting the following C data structure:

```
typedef struct e_s e_t;
struct e_s {
    char *str;
    int n;
    e_t **children;
};
```

where **str** is the identifier of the node of the tree, **n** is the number of children of the node, and **children** is the array of pointers referring to these children.

At the very beginning, the field **str** is defined only for the leaves of the tree, as represented by the leftmost tree of the following picture. For all internal nodes, **str** is not NULL but it is undefined (i.e., it points to an undefined, dynamically allocated, and useless string).



Write the function

```
<type> tree_complete (e_t *root);
```

which receives the tree rooted at **root** and it recursively completes the tree by assigning to each field **str** (but the ones on the leaves that have to remain untouched) a string which is the concatenation of the strings stored in the node's children. The result of this operation for the leftmost tree of the picture should deliver the rightmost tree of the same picture. The **type** of the function **tree_complete**, i.e., **<type>**, can be selected by the candidate.

Please, report the entire C code of the function. Allocate and free all required dynamic fields avoiding memory leaks. Briefly explain (in plain English language, within a C comment) the logic of the code.

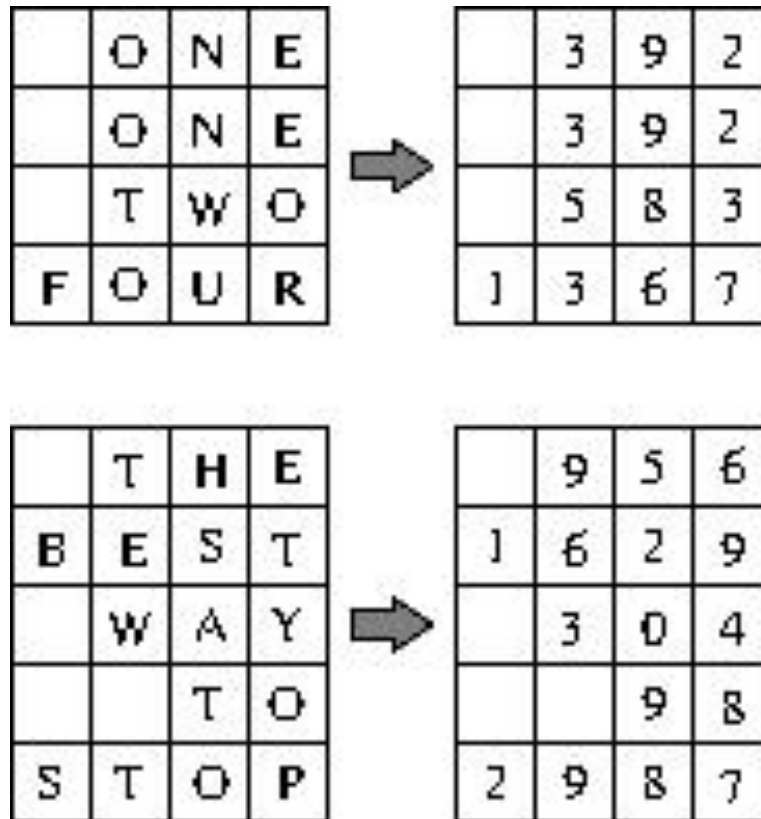
Domanda 3

Risposta non data

Punteggio max.:
14,00

An **alphametic square** is a peculiar type of mathematical puzzle, in which a set of words is written down in a matrix in the form of an ordinary addition sum, and it is required that the letters of the alphabet be replaced with decimal digits so that the result is a valid arithmetic sum.

The following picture reports two examples of alphametic squares. In the first one (the one on top), a matrix of size 4x4 indicates the following sum: "ONE"+"ONE"+"TWO"="FOUR". In the second one (the one at the bottom), a matrix of size 5x4 indicates the following sum: "THE"+"BEST"+"WAY"+"TO"="STOP".



The game consists of identifying for each letter the decimal digit satisfying the **operation**. The solutions for the two matrices are reported on the right-hand side of the picture. For example, in the first case, if we substitute the letter 'O' with the digit 3, the letter 'N' with the digit 9, etc., we obtain the top-right matrix. In this matrix, the corresponding numbers form a correct operation, i.e., $392+392+583=1367$. Similar considerations hold for the second example in which 'T' assumes the value 9, 'H' the value 5, etc., and the resulting sum is $956+1629+304+98=2987$.

Notice that the sum **must be performed in a standard way**, appropriately taking care of the carry values. For example, for the least significant digit of the last sum, $6+9+4+8$ is equal to 27, that is 7 with a carry equal to 2 which has to be moved and summed on the second digit of the number.

Write the function:

```
void alphametic_matrix (char **mat, int r, int c);
```

which receives the matrix **mat** including the alphametic square with its number of rows **r** and columns **c**, and it generates a solution (if a solution does exist). **If a solution exists, the function must display all letter-to-digit correspondence.**

To write the function, it is possible to make the **following hypothesis**:

- 1) The only operation performed on all numbers is the addition and the result is always represented in the last row of the matrix.
- 2) The input matrix includes only upper case letters (26 in total, i.e., 'A'-'Z').

- 3) Each letter is associated with a single and unique decimal value (10 digits, i.e., 0-9).
- 4) The strings do not necessarily have the same length. Thus, spaces are associated with the decimal value zero.
- 4) The result has a number of digits which is dependent on the addition performed on the previous numbers.
- 5) Strings in the same matrix cannot include more than 10 different characters (to map on 10 different digits, at most).

Please, report the entire code for the procedure **alphametic_matrix** (and all functions called by it). Explain (in plain English language) the logic of the code (e.g., the combinatorics principle that has been used).
